The Essential of Chapter 3

Indexing in four pages: the levels of the tree, the five page reads, the leading column, pages not rows, the price of an index

Advanced Data Management

2026-10-31

Numbers to carry: page in `shared_buffers` 0.1 μs, random SSD read 100 μs, HDD 10 ms; planner: random page 4, sequential page 1; `orders` 500,000 rows / 3,185 pages (157 per page), `customers` 10,000 rows / 73 pages.

1 An index beside the heap

Concept: Index, row id, selectivity

An **index** is a separate sorted structure mapping key values to **row ids** (the (page, slot) `ctid`s) of the heap, which stays as it is. **Selectivity** s of a predicate = fraction of rows it keeps: `id = 4275` is 2×10^{-6}; `amount > 20` is 0.85. An index makes matching reads fast, costs every write, and takes space.

Mock drill

Quiz A21–A23: the definition, selectivity as a fraction, and the sentence that opens **Mock 1, 2.3** and **Mock v2 C4**: "an index scan costs about one random page read per matching row, so past a few percent of the table the optimiser scans".

Key insight. Having an index does not mean the planner uses it: it costs the index path and the sequential scan *in pages* and takes the cheaper one.

2 B$^+$ tree: levels and the cost of one lookup

Mechanism: Structure

Each node is **one page** holding hundreds of (key, pointer) pairs: the **fanout** F (8-byte key + 8-byte pointer = 16 B, 8192/16 = 512; headers and two-thirds fill bring it to 100–300, hence 200 or 250 in the mocks). **Internal nodes** hold routing keys only (one more pointer than keys); **leaves** hold (key, row id) sorted, every key in exactly one leaf, **linked left to right** so a range walks forward. All leaves at the same depth.

PostgreSQL's `btree`.

Mechanism: Levels and lookup cost

$$\text{levels} = \lceil \log_F N \rceil \quad \text{(bracket } N \text{ between two powers of } F\text{)},$$
$$\text{point lookup} = \text{levels} + 1 \text{ (the heap page)}.$$

Count from the leaves too: N/F leaf pages, $/F$ again per level, until 1; the sum is the index size. An absent key costs h (no heap page). A range: h + leaves walked + one heap page per matching row. `ORDER BY k LIMIT 10` needs no sort (walk the leaf chain). A hash index serves equality only.

Mock 1, 1.4: 2,000,000 rows, $F = 200$. $200^2 = 40{,}000 < 2 \times 10^6 < 200^3 = 8 \times 10^6$: 3 levels; leaves 10,000 → 50 → 1. Lookup $3 + 1 = \mathbf{4}$ (b); the trap is (d) $21 = \log_2 2\text{M}$.
Mock v2 B2(a): 20,000,000 rows, $F = 250$. $250^3 = 15.6\text{M} < 2 \times 10^7 < 250^4$: 4 levels; leaves 80,000 → 320 → 2 → 1; index ≈ 80,323 pages ≈ 630 MB. Lookup $4 + 1 = \mathbf{5}$ page reads.
Mock v2 B2(b)(c), the 1 ms target: cold SSD $5 \times 100\ \mu\text{s} = 0.5$ ms (just under, no margin); cold HDD 5×10 ms $= 50$ ms, missed 50×. Root and level 2 resident (every lookup hits them): $2 \times 0.1 + 3 \times 100 \approx 0.3$ ms on SSD; HDD $3 \times 10 = 30$ ms, still 30× over. **Decision:** SSD; no cache saves an HDD whose single read is 10 ms; with a few GB of `shared_buffers` the whole 630 MB index is resident and only the heap page is read: 0.1 ms.

Mock drill

Mock 1, 1.4 / Mock v2 B2(a): bracket N between two powers of F, count from the leaves, "+1 for the heap page". **B2(b)(c)**: reads × device latency against 1 ms, what resident levels save, HDD fails on one read. **Mock v2 D2**: index nested loop for `c.id = 42` costs $1 + (3 + 1)$ pages against a 3,185-page hash-join probe.

3 Composite indexes: the leading column decides

An index on `(cust_id, order_date)` is sorted by `cust_id`, then by `order_date` *within* each customer (surname, then first name). A predicate can descend only if it fixes a **leading prefix**.

Predicate on `(cust_id, order_date)`	Uses it?	Why
`cust_id = 7`	yes	leading column; the customer's entries are contiguous
`cust_id = 7 AND order_date > '2025-06-01'`	yes, fully	descend on the equality, walk the contiguous date range
`order_date = '2025-03-01'`	no	not a leading column: one entry per customer, scattered
`cust_id + 0 = 7`	no	an expression hides the column
`ORDER BY order_date LIMIT 10`	no	ordered by date only within a customer
`cust_id > 5 AND order_date = ...`	partially	range on the leading column; date is only a filter

Design rules. Equality columns first, the range column last. **No function on the column**, or index the function: `CREATE INDEX ON users(lower(email))`. **Partial index** for a narrow fixed predicate: `ON events(user_id) WHERE event_type = 'purchase'` (0.5% of rows, 200× smaller, untouched by other inserts). **Covering index**: `ON orders(cust_id) INCLUDE (amount)` gives an index-only scan, no heap reads at all.

Mock drill

Mock v2 A3: (b), "sorted by `cust_id` first, then `order_date` within each customer; descend on the equality and walk the range", one clause each against (a) no leading column, (c) expression, (d) order on the second column. **Mock 1, 3.3(a)(b)(c)(e)(f)**: unique B-tree on `email`; B-tree on `ts` for the range (BRIN if appended in time order); partial on `user_id WHERE event_type='purchase'`; expression `lower(email)`; B-tree on the foreign key. **Quiz B4**: `city='Fes'` cannot use `(country, city)`; `country > 'M'` can.

4 When an index helps: pages, not rows

Mechanism: The two costs the planner compares

Seq Scan: B pages, sequential, streaming. Index Scan on a secondary index: h + leaves + **one random heap read per matching row**, charged 4 each: $\approx 4\,sN$, re-reading the same pages many times. With r rows per page, a page holds sr matches on average and is empty of matches with probability $(1-s)^r$, so the index scan visits $B\,(1-(1-s)^r)$ pages: once $sr \geq 3$ or so, **every page**, in random order.

Two thresholds. Plain index scan wins only when $4sN < B$, i.e. $s < 1/(4r)$ (a fraction of a percent). A **Bitmap Index Scan + Bitmap Heap Scan** sorts the row ids by page and reads each page once in order (no random penalty, no re-reads, but still every page with a match): stretches the crossover to a few percent. Above that: Seq Scan.

Correlation (`pg_stats.correlation`, -1 to 1): how closely physical order

follows the column. Near ±1 (append-only by time, or after `CLUSTER`) a range's rows sit in adjacent pages, so s of the rows is $\approx s$ of the pages read in order, and a **BRIN** index (per-block-range min/max) skips the rest for almost nothing. Near 0 the formula above applies.

Mock v2 C4: index on `order_date`, correlation -0.0015, query keeps 42,299/500,000 = 8.5%. Matches per page ≈ 13; $P(\text{none}) = (0.915)^{157} \approx 10^{-6}$: an index scan visits all 3,185 pages in random order, $3{,}185 \times 4 = 12{,}740$ against 3,185, plus ≈ 120 leaf pages; a bitmap scan still reads all 3,185. **No**; keep the Seq Scan. It would help only with correlation near 1 (271 pages, BRIN).
Mock 1, 1.2: 5,000 pages, 100 rows/page, 40% kept: $200{,}000 \times 4 = 800{,}000$ against 5,000; 40 matches on every page. Seq Scan (b); plain index pays only below $1/(4 \times 100) = 0.25\%$.
Mock 1, 3.3(d): `event_type='view'` is 90% of 50M rows on 1M pages: 45M random fetches against 1M sequential; 45 matches per page. No index helps.

Mock drill

Mock v2 C4 in the rubric's order: fraction, correlation sentence, matches per page, $(1-s)^r$, pages read and the 4× penalty, bitmap does not save it, "no", the one case where it would help. **Quiz C3(b)–(d)**: $3 + 14 + 2{,}000$ against 25,000 for 2,000 matches; 302,000 against 25,000 for 10%; crossover near $B/N \approx 0.8\%$.

5 The price of an index

Trade-offs: What every index costs, used or not

Space: key + row id per row, ≈ 16 B for an integer key, 20–30 MB per million rows; competes for the buffer pool. **Write amplification**: each `INSERT` writes the heap page plus one leaf page per index, $n + 1$ random page writes per row (Lab 3: 2–4× slower inserts with three indexes); an `UPDATE` of an indexed column deletes and re-inserts the entry. **Optimiser effort**: more paths to cost, more room for a bad estimate.

Never used, by selectivity: a column with few distinct values (`status`, a boolean, `event_type` 90% `'view'`) has $s \geq 0.1$ for most predicates, far above $1/(4r)$; the planner correctly ignores the index and it pays full write and space cost for nothing. **Deciding:** start from the workload (what is filtered, joined, sorted on, and how many rows come back); keep primary keys, foreign keys, join columns; one composite index by the leftmost-prefix rule rather than many single ones; partial indexes for narrow predicates; `EXPLAIN (ANALYZE, BUFFERS)` before and after; drop what shows `idx_scan = 0` in `pg_stat_user_indexes`.

Mock drill

Mock 1, 2.3 under five headings: reads, writes $(n+1)$, space, optimiser, "never used" with the $1/(4r)$ threshold and the `event_type` example, then the deciding list ending in `idx_scan = 0`.

Warning / Gotcha: The four mistakes that cost points

- $\log_2 N$ for the height: it is $\log_F N$, 3 or 4 levels, never 21; and a lookup is levels +1.
- Counting rows instead of pages: 8.5% of rows at correlation 0 is 100% of pages in random order. Say "matches per page" and $(1-s)^r$.
- "I created an index, so the query uses it." Only below a fraction of a percent (a few with a bitmap scan).
- Expecting `(cust_id, order_date)` to serve `order_date` alone, or `email` to serve `lower(email)`.

Summary: The essential in five lines

1. An index maps keys to row ids beside the heap; selectivity is a fraction; having one does not mean using it.
2. B^+ tree: one page per node, routing keys inside, chained sorted leaves. Levels $= \lceil \log_F N \rceil$; lookup = levels +1: 2M at 200 = 4, 20M at 250 = 5; 0.5 ms cold SSD, 0.3 ms with top levels resident, 50 ms HDD.
3. Composite index: sorted by the first column, then the second within it; only a leading prefix descends; equality first, range last; no function on the column.
4. Index scan = 4 per match and every page holding a match, $B(1-(1-s)^r)$; at correlation 0 a few percent of rows is every page; bitmap stretches the crossover to a few percent; correlation near 1 or BRIN makes ranges cheap.
5. Every index costs space, $n+1$ writes per insert, optimiser effort; index what the workload filters on and returns few rows; drop `idx_scan = 0`.